
Parallel 2D Image Convolution on MultiCore CPUs Using Java Threads

CENG-479 Parallel Programming

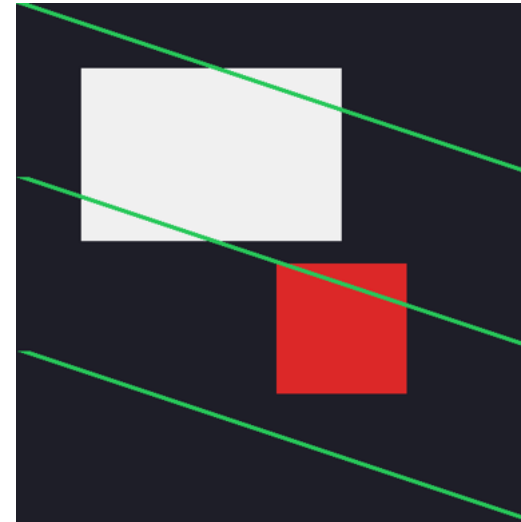
Team: Muhammet Akif Turan, Dafina Peci

Instructor: Asst. Prof. Dr. Huseyin Temucin

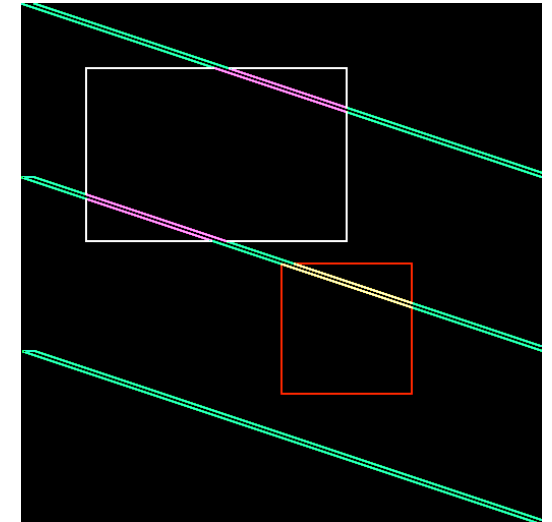
URL: <https://github.com/akifturan/CENG479-project/>

Problem & Motivation

- **2D image convolution:** slide a $k \times k$ kernel over every pixel (Gaussian blur, Sobel edge detection, sharpening, box blur)
- Work scales as $O(H \times W \times k^2 \times 3)$ per image
- Example: a 15×15 kernel on an 8192×8192 image $\approx$ **45 billion operations** — seconds on one CPU core
- Every output pixel is independent $\rightarrow$ "**embarrassingly parallel**" $\rightarrow$ ideal for multithreading



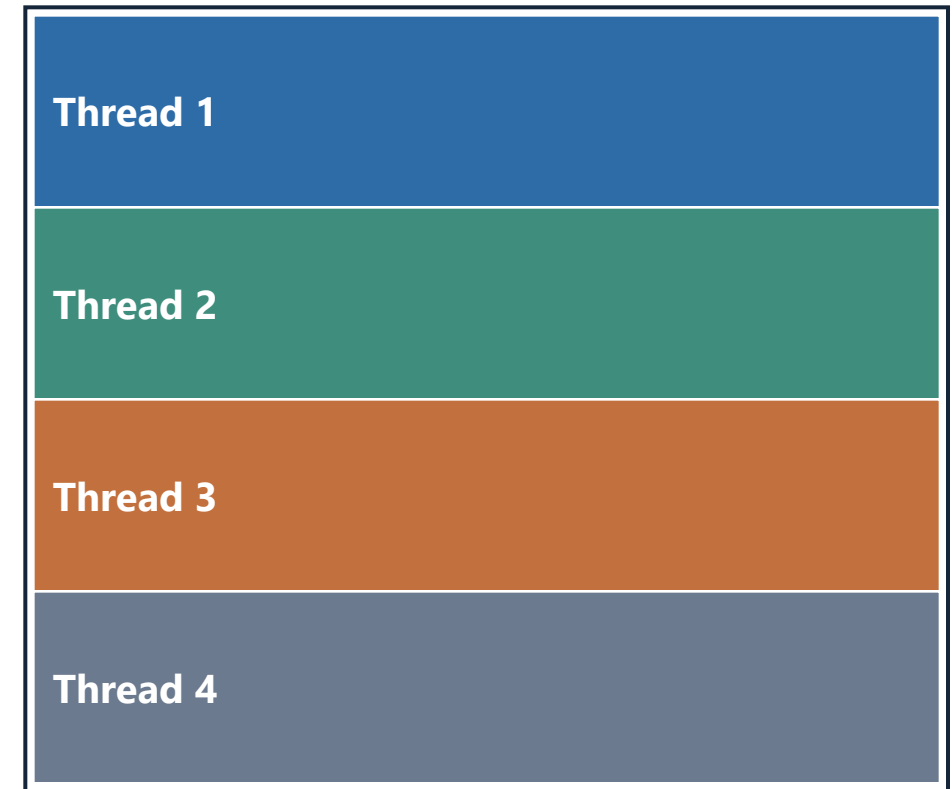
Input image



Sobel edge output

Approach: Row-Wise Parallelism

- Split the output image into **N horizontal row stripes**, one stripe per thread (domain decomposition)
- **Java ExecutorService** with a fixed-size thread pool runs the stripes
- Threads read the shared input and write to disjoint rows → **no locks, no false sharing**
- One shared per-pixel routine used by both versions → **identical results guaranteed**



Output image split into N row stripes, one per thread

Sequential Baseline vs Parallel — and Correctness

Sequential baseline (T_1)

Single thread processes all rows; the reference time for measuring speedup.

Parallel version (T_n)

Rows split into stripes; each stripe is a Callable task submitted to the thread pool.

Correctness: parallel output is byte-for-byte identical to sequential for all filters at 2 / 4 / 8 threads → **12 / 12**
PASS (no race conditions).

Experimental Setup

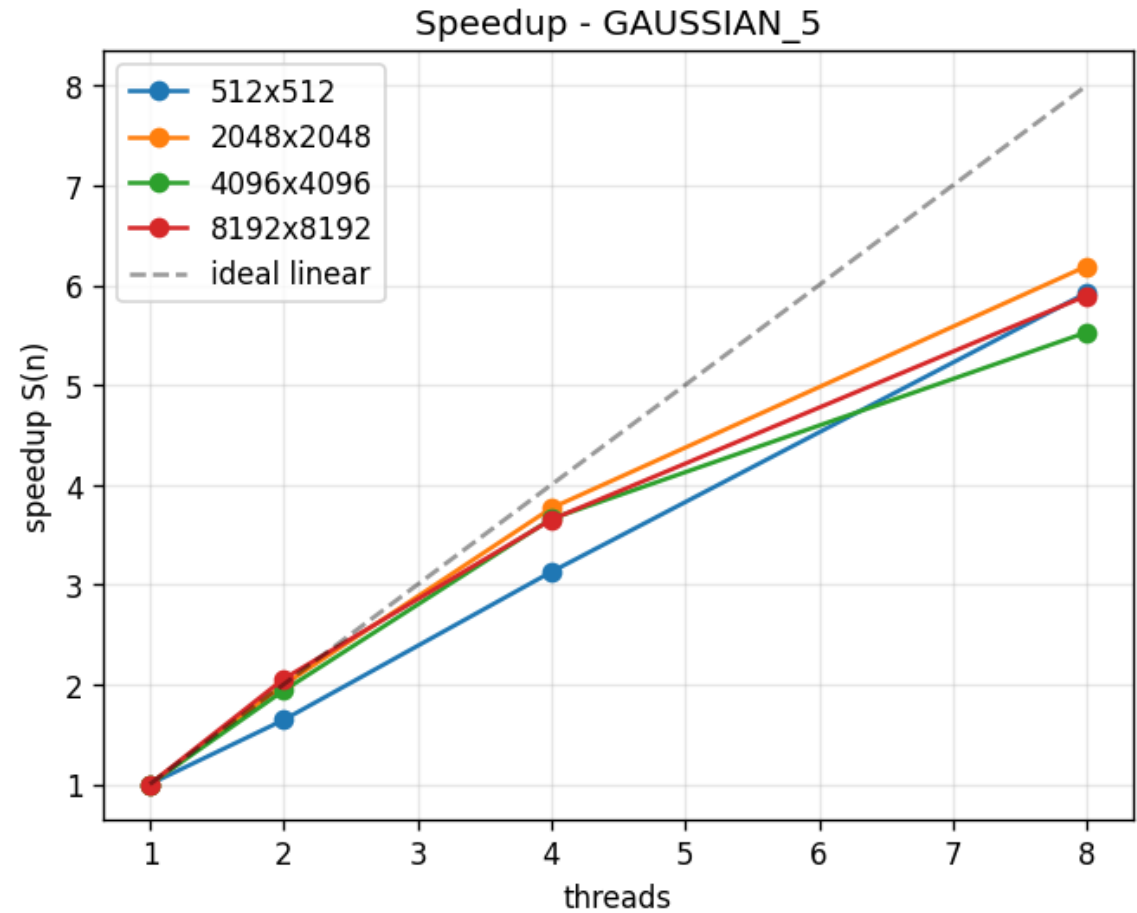
Dimension	Values	Cases
Filters	Gaussian 5×5 · Box 5×5 · Sharpen 3×3 · Sobel 3×3	4
Image sizes	512 ² · 2048 ² · 4096 ² · 8192 ²	4
Threads	1 , 2, 4, 8	4

$4 \times 4 \times 4 = 64$ benchmark cases + Gaussian kernel sweep ($k = 3, 5, 7, 15$)

- **Method:** 2 warm-up runs (JIT) + 5 timed runs, drop min/max (trimmed mean), image I/O timed separately
- **Two test machines:** Machine A = Intel i5-8265U (4C/8T, 15 W); Machine B = Intel i5-10300H (4C/8T)

Results: Speedup

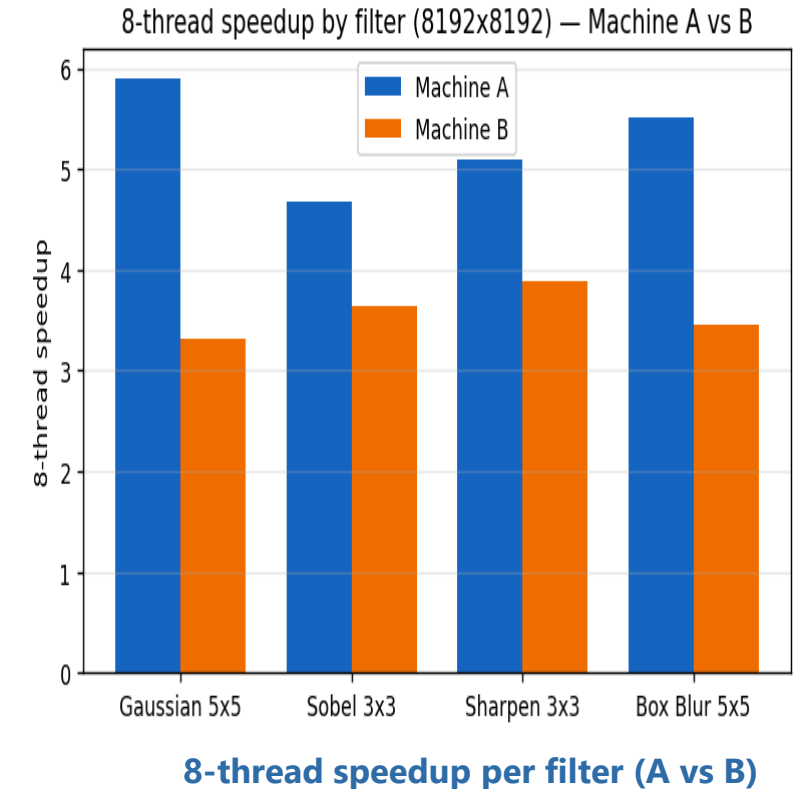
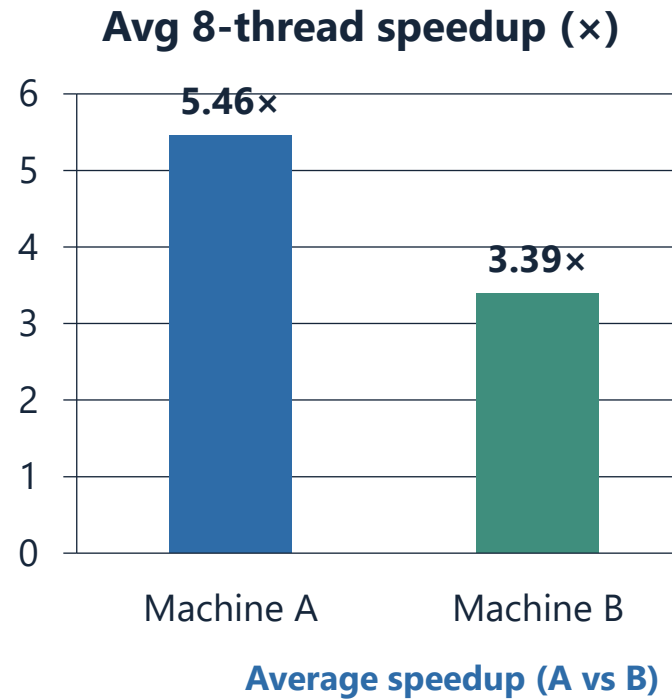
- **Near linear scaling** at low thread counts: $\sim 2\times$ at 2 threads, $\sim 3.2\text{--}3.8\times$ at 4 threads
- Speedup **flattens from 4 to 8 threads**, the CPU has 4 physical cores, the extra 4 are SMT (hyperthreading) siblings sharing execution units
- Best overall: **$6.43\times$ at 8 threads** (Gaussian 15×15)



Speedup vs threads

Hardware Matters: Machine A vs Machine B

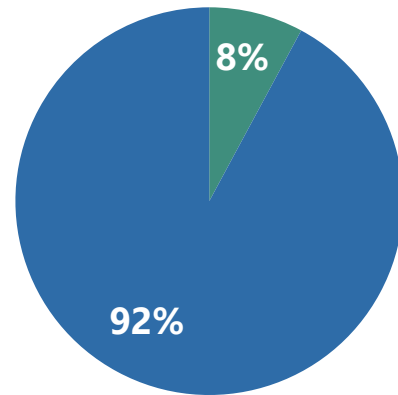
- Same code, two CPUs → **different speedup** (parallel performance is hardware-dependent)
- Average 8-thread speedup: **Machine A = 5.46x**, **Machine B = 3.39x**
- Differences come from CPU generation, power/thermal limits, cache and memory bandwidth



Kernel Size & Amdahl's Law

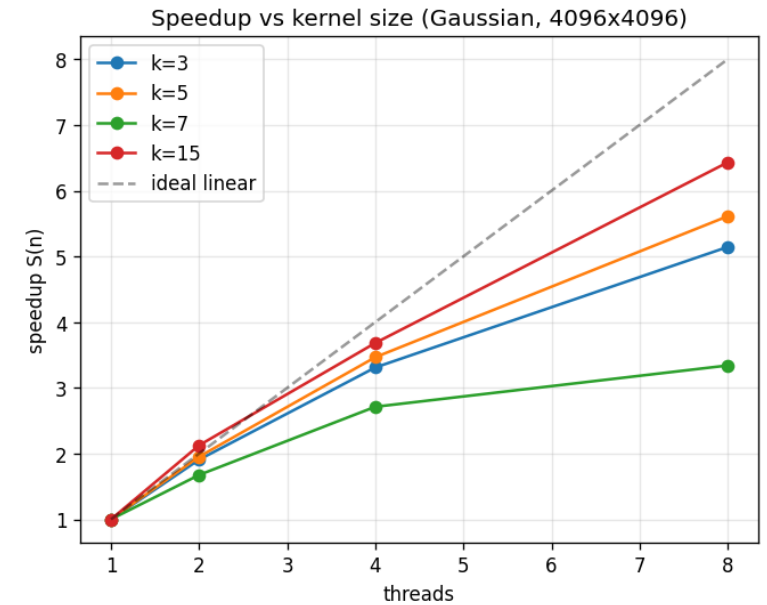
- **Larger kernels** = more compute per byte (higher arithmetic intensity) → scale better; 15×15 reaches the best **$6.43\times$ speedup**
- Image I/O is **not parallelized**, it is the fixed sequential fraction (Amdahl's Law)
- At 4096×4096 : 8-thread convolution $\approx$ **535 ms**, but PNG decode+encode $\approx$ **6269 ms** → I/O dominates end-to-end time

Time breakdown (ms)



■ Convolution (8T) ■ PNG I/O

Compute vs I/O time (4096^2)



Kernel-sweep speedup ($k = 3, 5, 7, 15$)

Challenges & Key Findings



Thermal throttling

15 W laptop CPU → reported speedup as a ratio (valid) + cooldown between heavy runs



JVM warm-up & noise

Warm-up runs (JIT) + trimmed mean over timed runs



Slow per-pixel access

Decode once into a primitive packed-RGB int array
→ better cache locality



Anomaly: 7×7 Gaussian

Scales worse than expected ($\sim 3.3\times$ at 8 threads), likely a cache/bandwidth interaction

Conclusion & Future Work

Conclusion

- Row-wise Java thread-pool parallelism gives solid speedup, **up to 6.43× (~80% of ideal)** on a 4-core / 8-thread CPU
- Results match theory: near-linear to 4 threads, SMT flattening after, then memory bandwidth & Amdahl (I/O) limits
- Correctness **guaranteed and verified automatically** (12/12 PASS)

Future Work

- ForkJoinPool / work-stealing comparison
- Separable Gaussian: $O(k^2) \rightarrow O(2k)$ per pixel
- Tile / block partitioning for cache reuse
- SIMD via the Java Vector API
- GPU port (CUDA) for large images